



算法与算法方法- 有限状态机模型



- 输入一个字符串，判断其是否是合法的C语言标识符；
- 输入一个字符串，判断其是否是 `abc` 形式(即先输入a、再输入b、最后输入c，且输入的a、b、c的个数相同)；
- 读取C源代码，去除注释；
 - 读取一行C语句，去除其中的注释并打印；
 - 分析：
 - 循环读取字符，直到换行；
 - 识别//，将其后的字符都丢掉不打印。
 - 识别/*...*/，将其中的字符都丢掉不打印。
 - 其他字符，原样打印。

解题思路：切分过程

```
int main(void){
    int count1=0,count2=0,count3=0;
    char ch;
    ch=getchar();
    while (ch=='a'){
        count1++;
        ch=getchar(); }
    while (ch=='b'){
        count2++;
        ch=getchar(); }
    while (ch=='c'){
        count3++;
        ch=getchar(); }
    if (ch=='#') {
        if ( (count1==count2)&&(count2==count3) )
            printf("string is acceptable!\n");
        else
            printf("string is not acceptable!\n");
    }
    return 0;
}
```

测试结果：

1、合法串可以识别。

#

abc#

aabbcc#

aaabbbccc#

...

2、能给出识别结果的错误串，仅有数量不等的串：

a#,b#,c#, aabcc#,

ab#,ac#,bc#,

3、不能给出识别结果的错误串：

不加#的，abc

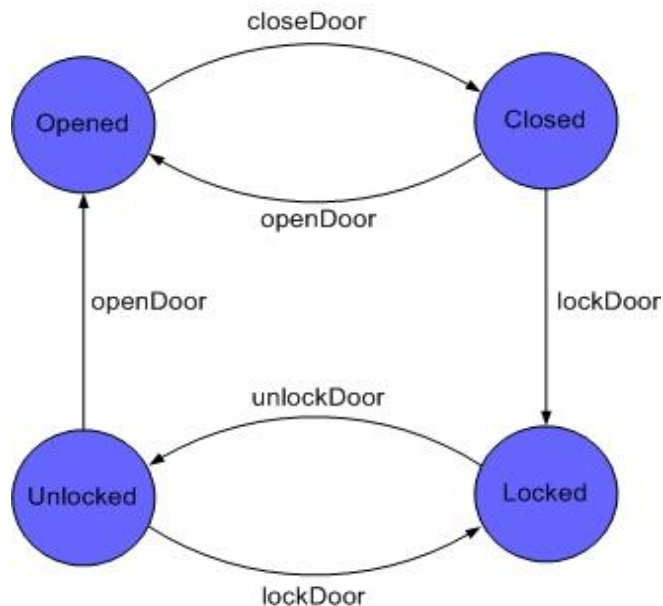
不按次序的，cba#,ba#,ca#,cb#,

- 对于连续判断问题, 循环+选择结构会导致逻辑路线模糊不清.
- 因此, 我们需要一种新的图形模型, 帮助我们整理连续判断问题的控制逻辑.
- 怎样才能把环环相扣的逻辑分开?
 - 使用状态变量, 记录之前发生的情况;
 - 在当前状态, 只处理当前规则; 后续的判断处理交给下一个状态。
- 针对类似的字符串识别、处理问题, 建立状态机模型, 可以为分析、求解带来很大的帮助。

什么是有限状态自动机？

有限自动机 (Finite Automata Machine) 是计算机科学的重要基石，它在软件开发领域内通常被称作有限状态机 (Finite State Machine)，是一种应用非常广泛的软件设计模式 (Design Pattern)。

有限状态机可以简称状态机，是表示有限个状态以及在这些状态之间的转移和动作等行为的数学模型。



状态：存储关于过去的信息，它反映从系统开始到现在时刻的输入变化。

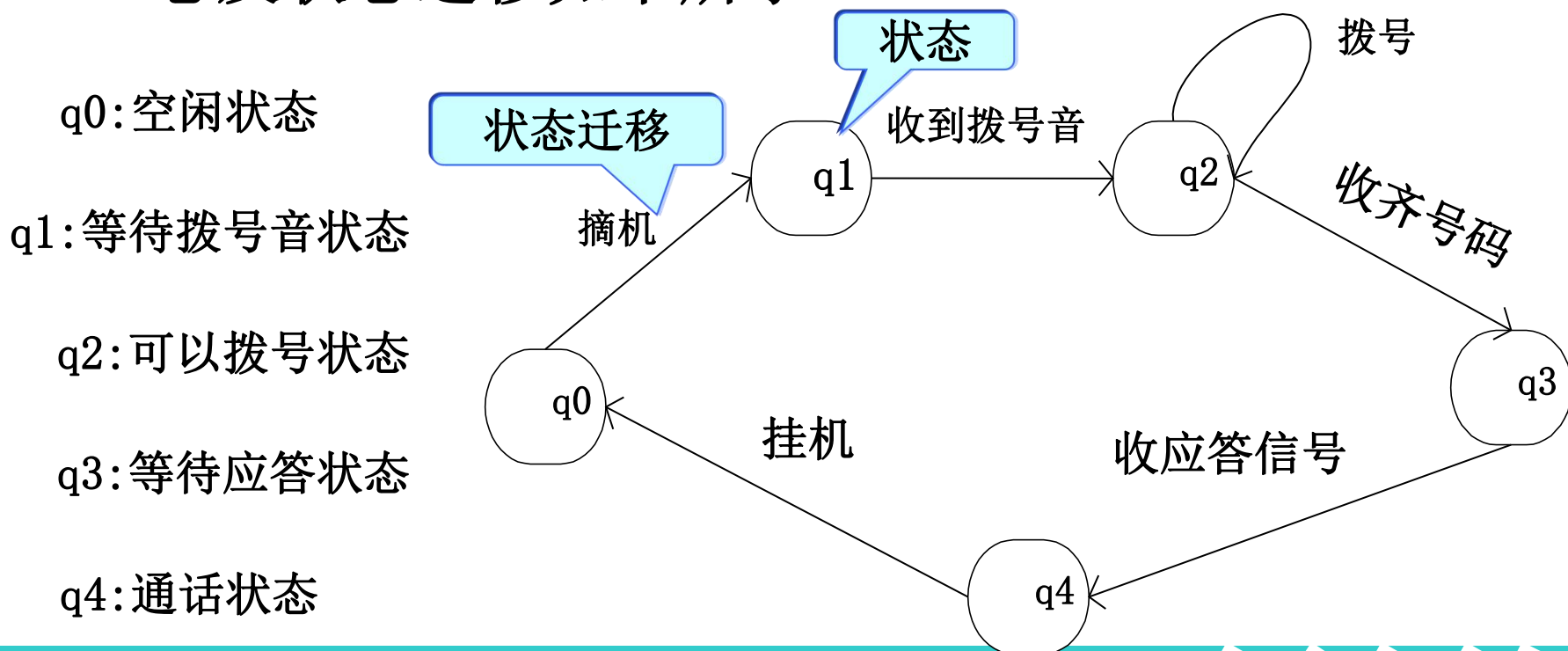
转移：指示状态变更，并且用必须满足来确使转移发生的条件来描述它。

动作：在给定时刻要进行的活动的描述。

4.7 基本概念

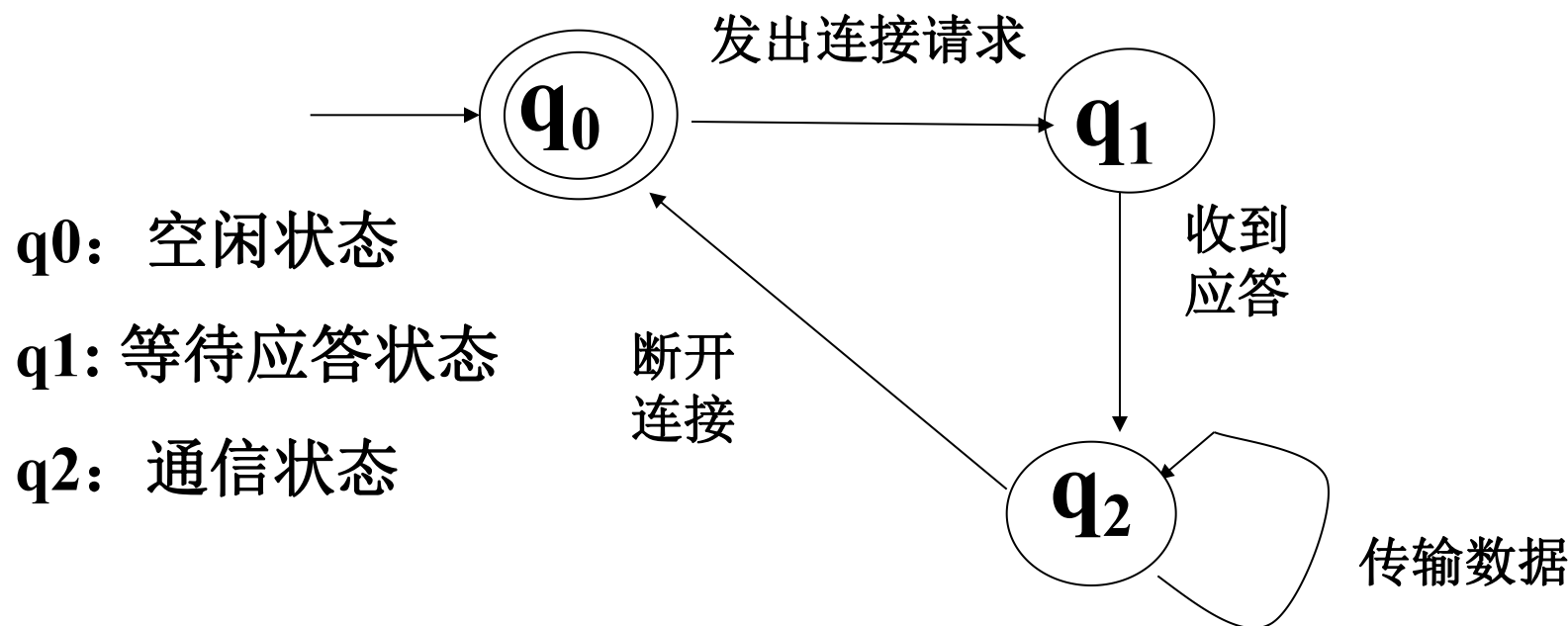
例1：打电话 (自动机在通信领域的应用)。

在一次呼叫中，从建立连接到通话完毕，要经历摘机，拨号，应答，进行通话等过程，话机的状态及状态迁移如下所示。

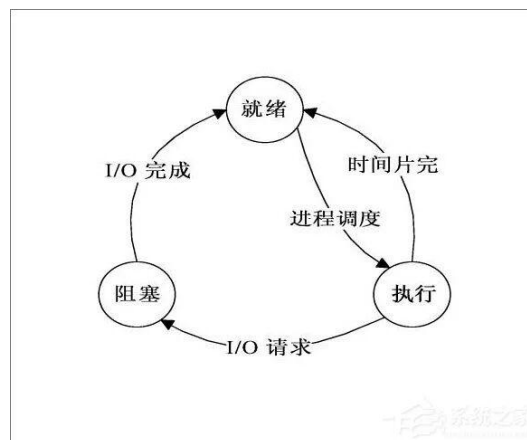


例2：串口通信（通信协议处理）

两台微机通过串口通信, 需在两台机器间建立好连接后, 才可以传递数据, 可以使用有限状态自动机, 描述串口通信的状态。



- 事实上许多实用的软件系统都必须维护一两个非常关键的对象，它们通常具有非常复杂的状态转换关系，而且需要对来自外部的各种异步事件进行响应。
 - 例如，在VoIP电话系统中，电话类（Telephone）的实例必须能够响应来自对方的随机呼叫，来自用户的按键事件，以及来自网络的信令等。在处理这些消息时，Telephone所要采取的行为完全依赖于它当前所处的状态，因而此时使用状态机就将是一个不错的选择。
 - 例：操作系统的进程调度



- 确定的有限自动机的形式化定义

确定的有限自动机是一个五元组

其中：

- ：有限的状态集合；
- ：有限的输入字母表；
- ：转换函数，是 到 的映射；
- ：初始状态， ；（初始状态只有一个）
- ：终止状态集， ；

- 如何设计有限状态自动机？
- 可能出现的有限个状态、运行的规则都是事先确定的。一旦开始运行，就按照事先确定的规则工作，因此叫“自动机”。
- 使用状态转换图形直观描述整个工作过程。

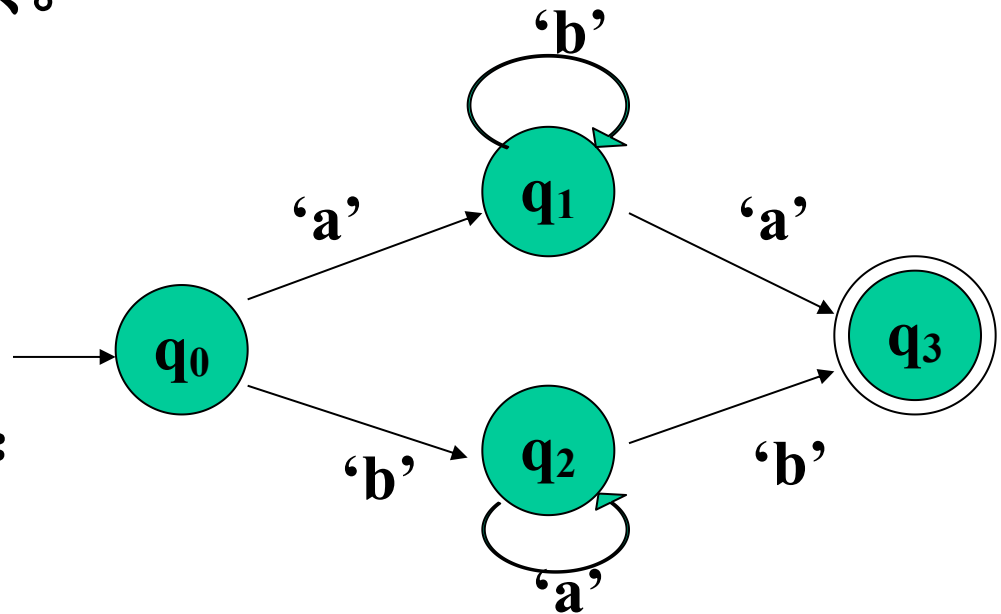
4.7 基本概念

- 为了描述一个有限状态自动机的工作状况，可采用**状态转换图**。状态转换图是一个有向图，图中的每个节点表示一种状态，一条边（或弧）表示一个转换关系。

初始状态： q_0

终止状态： q_3

控制器的状态集合：
 $\{q_0, q_1, q_2, q_3\}$



6.2 程序设计实例研究

设计交通车辆观测统计算法。

问题描述：在一个路口设置一个探测器，通过通信线路连接到后台的计算机。路口每通过一辆汽车，探测器向计算机发出一个车辆信号‘1’，探测器每隔1秒钟向计算机发出一个时钟信号‘0’，观测结束向计算机发出结束信号‘#’。

要求在计算机上设计一个程序，能够接收探测器发出的信号，统计出观测的时长、在观测时长内通过的车辆总数、以及两辆车之间最大的时间间隔。

分析：观测时长=字符序列中0的个数(6秒)；

车辆总数=字符序列中1的个数(9辆)；

两车间最大时间间隔=两个1之间的最大连续0的个数(3秒)；

q0:初始状态

q1:读入字符'1'即进入q1状态,
说明前一个字符是 '1'

q2:读入字符'0'即进入q2状态,
说明前一个字符是 '0'

q3:终止状态

转换规则

- 初始状态是q0 (state==q0):
 - 读入'1': `vehicles++`; `state=q1`
 - 读入'0': `seconds++`; `state=q2`
 - 读入EOF: `state=q3`
- 当前状态是q1 (state==q1, 说明前一个字符是'1'):
 - 读入'1'(11): `vehicles++`;
 - 读入'0'(10): `interval=1`; `seconds++`; `state=q2`
 - 读入EOF: `state=q3`



- 当前状态是q2 ($state == q2$, 说明前一个字符是'0'):
 - 读入'1' (01): $if(vehicles > 0)$ 处理最大时长
 $vehicles++$; $state = q1$
 - 读入'0' (00): $seconds++$;
 $if(vehicles > 0) interval++$;
 - 读入EOF: $state = q3$

终止状态集 $F = \{q3\}$

有限状态机的通用模式

```
#define START 1  
  
...  
int state=START;  
  
...  
while (state!=END) {  
    ch=getchar();  
    switch(state) {  
        case START:  
            if (ch==?)  
                state=Q1; break;  
        ...  
    }  
}
```

有限状态机解题通用处理模式

```
#include<stdio.h>
#define START 0
#define GET1 1
#define GET0 2
#define END 3

int main(void)
{
    char signal;
    int vehicles,seconds,interval,longest;
    int state;

    state=START;
    vehicles=0; seconds=0;longest=0;
    printf("input signals,'#' to end:\n");
```



```
while(state!=END){
    signal=getchar();
    switch(state){
        case START:/*当前是初始状态，则简单对时钟数或者车辆数加1*/
            switch(signal){
                case '1':
                    vehicles++;
                    state=GET1; /*状态迁移到1*/
                    break;
                case '0':
                    seconds++;
                    state=GET0;
                    break;
                case '#':
                    state=END;
                    break;
            }
        break;
    }
```



```
case GET1:/*若当前状态是q1，即上一个信号是'1'*/  
    switch(signal){  
        case '1': /*当前读入信号是'1'，则只是简单将车辆加1*/  
            vehicles++;    break;  
        case '0': /*前信号是'0'，则开始间隔interval*/  
            seconds++;    interval=1; /*开始间隔计数*/  
            state=GET0;    break;  
        case '#':  
            state=END;    break;  
    }  
    break;
```

```
case GET0: /*上一个信号是'0'*/
    switch(signal){
        case '0': /*当前读入'0',则判断是否将interval加1*/
            seconds++;
            if(interval>0) interval++;
            break;
        case '1': /*当前读入'1',则判断是否处理最长时间间隔*/
            vehicles++;
            if(interval>longest)
                longest=interval;
            interval=0;
            state=GET1;
            break;
        case '#':
            state=END;      break;
    }
    break;
```

```
        }/*switch*/  
    }/*while*/  
  
    printf("%d vehicles passed in %d  
seconds\n",vehicles,seconds);  
    printf("the longest gap was %d  seconds\n",longest);  
  
    return 0;  
  
}
```

/*任意01#串都可以测试，都有结果。
非法的字符串也有结果，只是不输入#就不能结束输入。
*/

6.2 程序设计实例研究

例2 检验输入字符串是否是合法的C语言注释

`/*...*/`

q1:等待注释开始的*

q2:等待注释结束的*

q3: 等待注释结束的/

q4: 等待EOF状态

导论教材P172,图10.10,注意读程序实例

6.2 程序设计实例研究

- 转换函数分析
 - start状态下:
 - 输入 ‘/’ :state=q1
 - 输出非 ‘/’ :state=ERROR
 - q1状态下:
 - 输入 ‘*’ :state=q2
 - 输出非 ‘*’ :state=ERROR
 - q2状态下:
 - 输入 ‘*’ :state=q3
 - 输入EOF: state=ERROR
 - 输出其他: state=q2

6.2 程序设计实例研究

- 转换函数分析(续)
 - q3状态下:
 - 输入 ‘*’:状态不变
 - 输入 ‘/’ :state=q4
 - 输入EOF: state=ERROR
 - 输出其他: state=q2
 - q4状态下:
 - 输入EOF: state=ACCEPT
 - 输出其他: state=ERROR

总结：有限状态机解题模式

```
#define START 1  
  
...  
int state=START;  
  
...  
while (state!=END) {  
    ch=getchar();  
    switch(state) {  
        case START:  
            if (ch==?)  
                state=Q1; break;  
        ...  
    }  
}
```

有限状态机解题通用处理模式

```
#include<stdio.h>
#define ERROR -1
#define ACCEPT 1
#define Q1 2
#define Q2 3
#define Q3 4
#define Q4 5
#define START 0
```

```
#include <stdio.h>

int main()
{
    char ch;
    int state=START;
    while(state!=ACCEPT && state!=ERROR)
    {
        scanf("%c",&ch);
        switch(state)
        {
            case START: if(ch=='/') state=Q1;
                        else state=ERROR; break;
            case Q1: if(ch=='*') state=Q2;
                    else state=ERROR; break;
```

```
case Q2: if(ch=='*') state=Q3;
        if(ch=='\n') state=ERROR; break;
case Q3:
        switch(ch) {
        case '/': state=Q4;break;
        case '*': state=Q3;break;
        case '\n': state=ERROR; break;
        default: state=Q2; };break;
case Q4: if(ch=='\n') state=ACCEPT;
        else state=ERROR; break; }
}
if(state==ACCEPT) printf("Yes\n");
else printf("No\n");
```

练习：读入多行字符串，将其中的行注释和段注释去掉并打印其余字符。

